

compiler about this assumption using the `tailrec` annotation (<http://mng.bz/BWT5>), so it can give us a compile error if it's unable to eliminate the tail calls of the function. Here's the syntax for this:

```
def factorial(n: Int): Int = {
  @annotation.tailrec
  def go(n: Int, acc: Int): Int =
    if (n <= 0) acc
    else go(n-1, n*acc)
  go(n, 1)
}
```

We won't talk much more about annotations in this book (you'll find more information at <http://mng.bz/GK8T>), but we'll use `@annotation.tailrec` extensively.

NOTE See the preface for information on the exercises.



EXERCISE 2.1

Write a recursive function to get the n th Fibonacci number (<http://mng.bz/C29s>). The first two Fibonacci numbers are 0 and 1. The n th number is always the sum of the previous two—the sequence begins 0, 1, 1, 2, 3, 5. Your definition should use a local tail-recursive function.

```
def fib(n: Int): Int
```

2.4.2 Writing our first higher-order function

Now that we have `factorial`, let's edit our program from before to include it.

Listing 2.2 A simple program including the factorial function

```
object MyModule {
  ...
  private def formatAbs(x: Int) = {
    val msg = "The absolute value of %d is %d."
    msg.format(x, abs(x))
  }

  private def formatFactorial(n: Int) = {
    val msg = "The factorial of %d is %d."
    msg.format(n, factorial(n))
  }

  def main(args: Array[String]): Unit = {
    println(formatAbs(-42))
    println(formatFactorial(7))
  }
}
```

← Definitions of `abs` and `factorial` go here.